A PROJECT REPORT

ExamIQ: A Smart Examination and Performance Analyzer

(PL/SQL (Oracle Database))

Database schema, result view and reporting package

Submitted By:

<<STUDENT NAME>>

UID: <<UID>>

Subject: PL/SQL (Oracle Database)

Submitted To:

<<PL/SQL SUPERVISOR>>

in partial fulfillment for the award of the degree of

MASTERS OF COMPUTER APPLICATIONS

IN

UNIVERSITY INSTITUTE OF COMPUTING

CHANDIGARH UNIVERSITY

October-November, 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "ExamIQ: A Smart Examination and Performance Analyzer" submitted by <<STUDENT NAME>> (<<UID>>), of University Institute Of Computing, Chandigarh University, is a Bonafide record of the work carried out under my guidance in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications at Chandigarh University during the academic year 2026-2028.

The project work embodied in this report has not been submitted to any other University or Institution for the award of any degree or diploma.

_____

Signature of Supervisor
<<PL/SQL SUPERVISOR>>

Table of Contents

1. Abstract 4
2. Introduction 4
3. Objectives 5
4. Software & Hardware Requirements 6
5. Implementation 7
6. Key Functionalities 10
7. System Analysis & Differentiation 11
8. Flow Chart 12
9. Output & Screenshots 13
10. GitHub Upload 14
11. Conclusion 16
12. Future Advancements 16

Abstract

ExamIQ: A Smart Examination and Performance Analyzer stores its examination data in a relational database, and this report documents the PL/SQL layer that implements that storage. The layer consists of three database objects defined in a single deployment script (sql/oracle_setup.sql): the table EXAMIQ_STUDENTS, which holds one row per student with a generated identity key, a unique student code and five subject marks each constrained to the range 0 to 100; the view EXAMIQ_STUDENT_RESULTS, which derives the total, the percentage and the grade for every row so that no client has to repeat the arithmetic; and the package EXAMIQ_REPORTS, whose procedures CLASS_SUMMARY and SUBJECT_AVERAGE return the class-level statistics - student count, average percentage, highest total and pass rate - and the mean of any one subject. The design keeps the business rules inside the database rather than in the application. Constraints guarantee that a mark outside 0-100 can never be stored, the view guarantees that the grade rule is applied identically everywhere, and the package exposes the reporting logic as a named, reusable API with typed IN and OUT parameters. Access from the Python client is parameterised: every statement uses bind variables, writes are wrapped in a transaction that is committed explicitly, and the same adapter module can also address a local SQLite file so that the project remains demonstrable without a database server. The report closes with the verification script, the expected results on the sample dataset, and a comparison of set-based database processing against row-by-row computation in the application.

Introduction

An examination system has two kinds of rules. Some are structural - every student has exactly one code, and no mark may be negative or greater than the maximum - and some are computational, such as the grade that follows from a total. Both kinds belong in the database, because the database is the only component that every client shares. If the grade boundary is written in the user interface, a second client (the web variant of this project, for instance) will eventually disagree with it; if the range of a mark is enforced only by form validation, a direct INSERT will corrupt the data.

The PL/SQL submission therefore concentrates on three ideas. First, the table declares its own integrity rules with NOT NULL, UNIQUE and CHECK constraints. Second, the view EXAMIQ_STUDENT_RESULTS expresses the derived columns - total marks, percentage and

grade - as a single, reusable definition instead of repeating them in every query. Third, the package EXAMIQ_REPORTS packages the reporting procedures behind a stable interface, with the calculation implemented in SQL so that the server aggregates the rows and returns four scalars instead of transferring the whole table to the client.

The same three objects also make the project easier to demonstrate. The deployment script is idempotent for the view and the package (both are created with CREATE OR REPLACE), so it can be re-run while the logic is being refined, and the anonymous PL/SQL block included in the script prints the summary through DBMS_OUTPUT so that the procedure can be verified from SQL Developer or SQL*Plus without any application code at all.

Objectives

The database layer was designed against the following objectives:

1. **Guarantee data integrity at the storage level:** declare a primary key, a unique student code and a CHECK constraint of 0-100 on each of the five marks, so that invalid data is rejected even when it arrives from outside the application.
2. **Centralise the derived business rules:** define the total, percentage and grade rules once, in the view EXAMIQ_STUDENT_RESULTS, and use CASE expressions so that the thresholds are visible in a single place.
3. **Provide a reusable reporting API:** encapsulate the class summary and the subject average in the package EXAMIQ_REPORTS with typed parameters, so that reports can be produced from SQL Developer, from SQL*Plus or from the Python client without duplicating SQL.
4. **Process data set-wise on the server:** use SQL aggregation (COUNT, AVG, MAX) inside the procedures so that one call returns four scalars instead of transferring every row to the client for computation.
5. **Access the data safely from the application:** use bind variables for every value, commit explicitly after a write, and keep one adapter module capable of addressing either Oracle or a local SQLite file for demonstration purposes.

Software & Hardware Requirements

1. Software Requirements

- **Database Server:** Oracle Database 19c / 21c Express Edition (or any Oracle Database release that supports identity columns), accessed through the schema that owns the EXAMIQ objects.
- **Database Client:** Oracle SQL Developer for running sql/oracle_setup.sql and demonstrating the package; SQL*Plus can be used instead, with SET SERVEROUTPUT ON for the DBMS_OUTPUT output.
- **Language:** SQL (DDL and DML) and PL/SQL for the view, the package specification, the package body and the verification block.
- **Driver:** python-oracledb 2.x, installed with pip install oracledb, used by database.py in Oracle mode.
- **Application Runtime:** Python 3.10 or later; the environment variables DB_MODE=oracle, ORACLE_USER, ORACLE_PASSWORD and ORACLE_DSN select and authenticate the Oracle connection.
- **Demonstration Mode:** SQLite 3 through the standard-library sqlite3 module, which lets the application run without a database server while the Oracle objects remain the formal submission.
- **Development Environment:** SQL Developer, Visual Studio Code, the Oracle client libraries required by python-oracledb, and Git.

2. Hardware Requirements

- **Processor:** 2.0 GHz dual-core or better; the Oracle instance is the largest consumer of resources.
- **Memory:** 4 GB RAM minimum, 8 GB recommended when the Oracle instance and the desktop client run together.
- **Storage:** 10 GB free space for the Oracle installation; the project's own schema occupies a few megabytes.
- **Network:** a local listener on port 1521, addressed by a DSN such as localhost:1521/XEPDB1.
- **Display:** 1366 x 768 for the application that consumes the data; no display is needed for SQL*Plus.

Implementation

The deployment script is organised in four parts: the table definition, the result view, the reporting package and a set of sample rows with the verification block. The script is meant to be executed once, as the project schema user, from SQL Developer.

1. Table - EXAMIQ_STUDENTS

The table uses a generated identity column as the surrogate key, a unique business key, and a domain constraint on every mark. NUMBER(3) with a BETWEEN 0 AND 100 check enforces the mark range in the database itself, so neither the desktop client nor a manual INSERT can store an impossible value.

Listing 1: table definition with identity key, unique code and range constraints

```
CREATE TABLE examiq_students (
    student_id        NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    student_code      VARCHAR2(30)  NOT NULL UNIQUE,
    student_name      VARCHAR2(100) NOT NULL,
    mathematics       NUMBER(3) NOT NULL CHECK (mathematics      BETWEEN 0 AND 100),
    computer_science  NUMBER(3) NOT NULL CHECK (computer_science BETWEEN 0 AND 100),
    physics           NUMBER(3) NOT NULL CHECK (physics          BETWEEN 0 AND 100),
    english           NUMBER(3) NOT NULL CHECK (english          BETWEEN 0 AND 100),
    dbms              NUMBER(3) NOT NULL CHECK (dbms             BETWEEN 0 AND 100)
);
```

2. View - EXAMIQ_STUDENT_RESULTS

The view publishes the derived columns so that neither the interface nor a future report has to repeat the arithmetic. The grade rule is expressed as a searched CASE expression, which keeps all four bands in one readable block and guarantees that every query sees the same thresholds.

Listing 2: result view with total, percentage and grade

```
CREATE OR REPLACE VIEW examiq_student_results AS
SELECT student_id, student_code, student_name,
       mathematics, computer_science, physics, english, dbms,
       mathematics + computer_science + physics + english + dbms AS total_marks,
       ROUND((mathematics + computer_science + physics + english + dbms) / 5, 2) AS percentage,
       CASE
         WHEN mathematics + computer_science + physics + english + dbms >= 400 THEN 'A'
         WHEN mathematics + computer_science + physics + english + dbms >= 300 THEN 'B'
         WHEN mathematics + computer_science + physics + english + dbms >= 200 THEN 'C'
         ELSE 'Needs Support'
       END AS grade
FROM examiq_students;
```

3. Package specification and body - EXAMIQ_REPORTS

The package separates the public interface from the implementation. CLASS_SUMMARY returns the four class-level figures through OUT parameters in a single aggregation query, and

SUBJECT_AVERAGE maps a subject name supplied at run time onto the corresponding column with a CASE statement, raising a user-defined error when the name is unknown.

Listing 3: package specification

```
CREATE OR REPLACE PACKAGE examiq_reports AS
    PROCEDURE class_summary(
        p_student_count      OUT NUMBER,
        p_average_percentage OUT NUMBER,
        p_top_total          OUT NUMBER,
        p_pass_rate          OUT NUMBER);
    PROCEDURE subject_average(p_subject IN VARCHAR2, p_average OUT NUMBER);
END examiq_reports;
/
```

Listing 4: package body with set-based aggregation and a user-defined exception

```
CREATE OR REPLACE PACKAGE BODY examiq_reports AS
    PROCEDURE class_summary(p_student_count      OUT NUMBER,
                            p_average_percentage OUT NUMBER,
                            p_top_total          OUT NUMBER,
                            p_pass_rate          OUT NUMBER) IS
    BEGIN
        SELECT COUNT(*), NVL(AVG(percentage), 0), NVL(MAX(total_marks), 0),
               NVL(100 * AVG(CASE WHEN percentage >= 40 THEN 1 ELSE 0 END), 0)
        INTO   p_student_count, p_average_percentage, p_top_total, p_pass_rate
        FROM   examiq_student_results;          -- aggregates computed by the server
    END;

    PROCEDURE subject_average(p_subject IN VARCHAR2, p_average OUT NUMBER) IS
    BEGIN
        CASE UPPER(p_subject)
          WHEN 'MATHEMATICS' THEN SELECT NVL(AVG(mathematics),0) INTO p_average FROM
examiq_students;
          WHEN 'COMPUTER SCIENCE' THEN SELECT NVL(AVG(computer_science),0) INTO p_average FROM
examiq_students;
          WHEN 'PHYSICS' THEN SELECT NVL(AVG(physics),0) INTO p_average FROM examiq_students;
          WHEN 'ENGLISH' THEN SELECT NVL(AVG(english),0) INTO p_average FROM examiq_students;
          WHEN 'DBMS' THEN SELECT NVL(AVG(dbms),0) INTO p_average FROM examiq_students;
          ELSE RAISE_APPLICATION_ERROR(-20001, 'Unknown subject');
        END CASE;
    END;
END examiq_reports;
/
```

4. Verification block

The script ends with three sample rows and an anonymous block that calls both procedures and prints the result. The block is the fastest way to prove that the package works, because it needs no application, no driver and no user interface - only SET SERVEROUTPUT ON.

Listing 5: anonymous PL/SQL block used for verification

```
SET SERVEROUTPUT ON;
DECLARE
    v_count NUMBER; v_avg NUMBER; v_top NUMBER; v_pass NUMBER; v_dbms NUMBER;
BEGIN
    examiq_reports.class_summary(v_count, v_avg, v_top, v_pass);
    examiq_reports.subject_average('DBMS', v_dbms);
    DBMS_OUTPUT.PUT_LINE('Students            : ' || v_count);
    DBMS_OUTPUT.PUT_LINE('Average percentage  : ' || ROUND(v_avg, 2));
    DBMS_OUTPUT.PUT_LINE('Top total           : ' || v_top);
    DBMS_OUTPUT.PUT_LINE('Pass rate           : ' || ROUND(v_pass, 2) || '%');
    DBMS_OUTPUT.PUT_LINE('DBMS average        : ' || ROUND(v_dbms, 2));
```

```
END;
/
```

5. Application access - database.py in Oracle mode

The Python adapter keeps the Oracle branch symmetric with the SQLite branch: a cursor is opened, every value is passed as a bind variable, rows are fetched and mapped to dictionaries, and writes are committed explicitly. Column names differ between the two schemas (student_id/student_name in Oracle, id/name in SQLite), and the adapter absorbs that difference so that no screen in the application needs to know which database is active.

Listing 6: parameterised access from the Python client

```
def get_connection():
    if DB_MODE == "oracle":
        import oracledb
        return oracledb.connect(user=os.environ["ORACLE_USER"],
                                password=os.environ["ORACLE_PASSWORD"],
                                dsn=os.environ["ORACLE_DSN"])
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def save_student(student_id, code, name, scores):        # Oracle branch
    with get_connection() as conn:
        cur = conn.cursor()
        if student_id:                                    # UPDATE with binds
            cur.execute("UPDATE examiq_students SET student_code=:c, student_name=:n, "
                        "mathematics=:a, computer_science=:b, physics=:p, "
                        "english=:e, dbms=:d WHERE student_id=:id",
                        {"c": code, "n": name, "a": scores[0], "b": scores[1],
                         "p": scores[2], "e": scores[3], "d": scores[4],
                         "id": student_id})
        else:                                             # INSERT with binds
            cur.execute("INSERT INTO examiq_students(student_code, student_name, "
                        "mathematics, computer_science, physics, english, dbms) "
                        "VALUES(:c,:n,:a,:b,:p,:e,:d)",
                        {"c": code, "n": name, "a": scores[0], "b": scores[1],
                         "p": scores[2], "e": scores[3], "d": scores[4]})
        conn.commit()
```

6. Object map

The four objects form a single dependency chain: the table stores the facts, the view adds the derived values, the package reports on the view, and the application reads through the adapter.

Object	Type	Purpose
EXAMIQ_STUDENTS	table	One row per student; identity key, unique code, five constrained marks.
EXAMIQ_STUDENT_RESULTS	view	Adds total_marks, percentage and grade to every row.
EXAMIQ_REPORTS	package	Public reporting API: class summary and per-subject average.
CLASS_SUMMARY	procedure	Returns student count, average percentage, top total and pass rate.
SUBJECT_AVERAGE	procedure	Returns the mean of one named subject, or raises -20001.

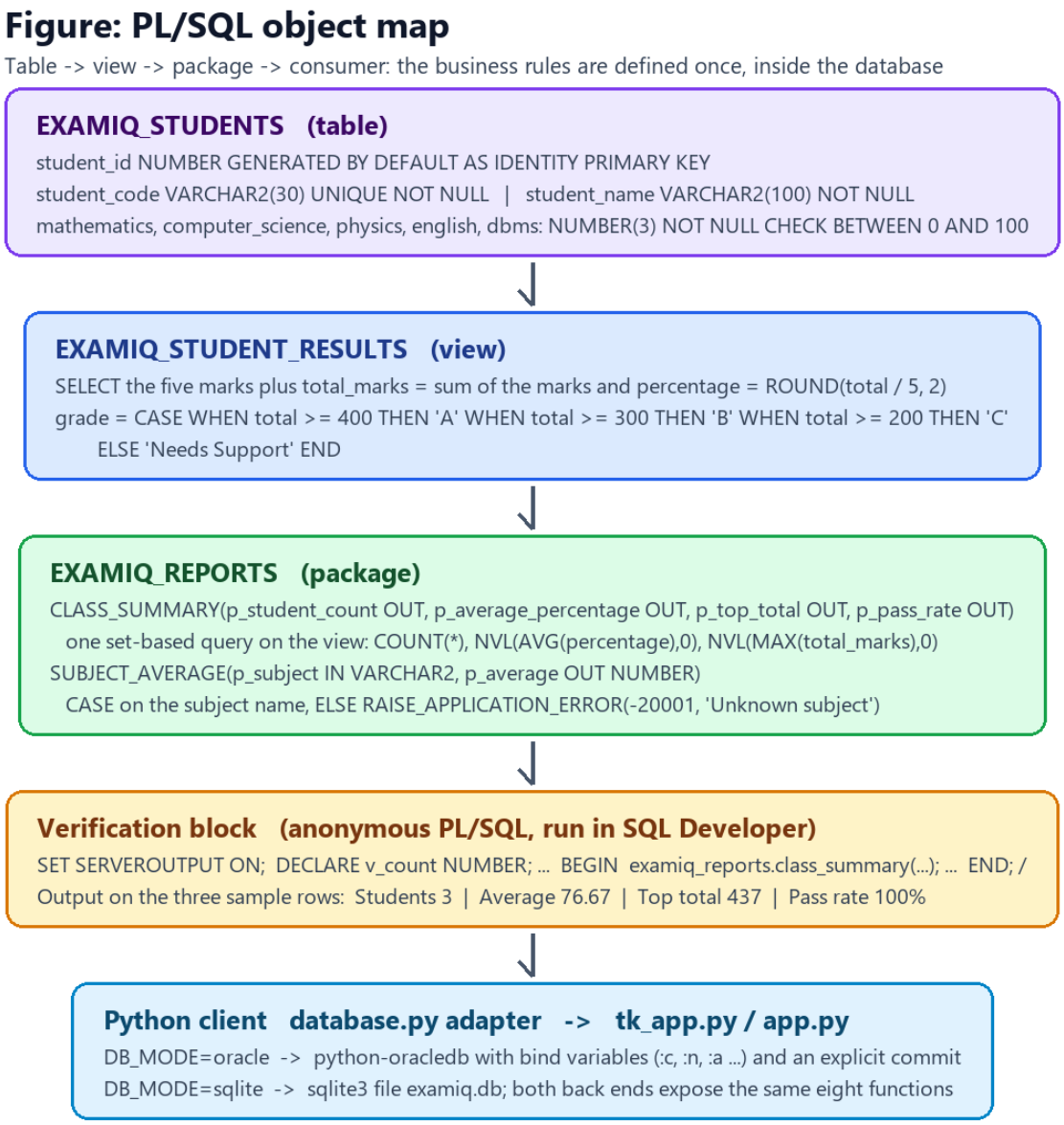


Figure 1: Object map of the PL/SQL layer and its consumer

Key Functionalities

1. Storage and integrity

- **Surrogate and business keys:** student_id is generated by the database, while student_code carries a UNIQUE constraint so that a code can never be used twice.
- **Domain constraints:** each mark column is NUMBER(3) NOT NULL with CHECK BETWEEN 0 AND 100, so an out-of-range or missing mark is rejected by the server.
- **Mandatory attributes:** student_name is NOT NULL, which prevents anonymous rows in reports and rankings.

2. Derived results

- **Single definition of the rules:** the view computes total_marks, percentage (rounded to two decimals) and the grade band A / B / C / Needs Support.
- **Query convenience:** reports can select from the view directly - SELECT * FROM examiq_student_results ORDER BY total_marks DESC - without repeating the arithmetic.
- **Consistency with the application:** the thresholds in the CASE expression match the grading function used by the Python layer, so both routes produce the same grade.

3. Reporting package

- **CLASS_SUMMARY:** one aggregation query returns the number of students, the average percentage, the highest total and the pass rate, using NVL to convert the empty-set NULLs of an empty table into zeros.
- **SUBJECT_AVERAGE:** accepts a subject name at run time and returns its mean, with RAISE_APPLICATION_ERROR(-20001, 'Unknown subject') for an invalid name.
- **Set-based processing:** the calculations run inside the server; the client receives four scalars instead of a result set, which is the main performance argument for PL/SQL in this project.
- **Idempotent deployment:** the view and the package use CREATE OR REPLACE, so the script can be re-run safely during development.

System Analysis & Differentiation

- **Rules in the database instead of the client:** constraints and the result view apply equally to the desktop client, the Flask variant and a manual INSERT, which removes a whole class of inconsistency between interfaces.
- **Set-based versus row-by-row computation:** the package aggregates with COUNT, AVG and MAX in one statement, whereas the demonstration adapter computes the same figures in Python after fetching the rows; the first approach moves far less data and is the reason the package exists.
- **A named API instead of ad-hoc SQL:** the procedures give the reporting logic a stable signature, so the implementation can be optimised without changing a single caller.
- **Defence in depth:** the same rule is enforced three times - in the form, in the adapter and in the constraint or view - which is cheap for this data volume and makes the system robust against misuse.
- **Portability with a single adapter:** Oracle and SQLite are addressed through the same eight functions, so the project can be demonstrated in a laboratory without a database server and submitted with the full Oracle schema.

- **Explicit transaction control:** writes are committed by the adapter after a successful statement, and the SQLite context manager rolls back automatically on error, so a partially written record is never visible.

Flow Chart

The flowchart shows how the application, the adapter and the PL/SQL objects interact for a typical request: a record is validated in the interface, written through the adapter with bind variables, and then reported either by a query on the view or by a call to the package.

Figure: ExamIQ process flow chart

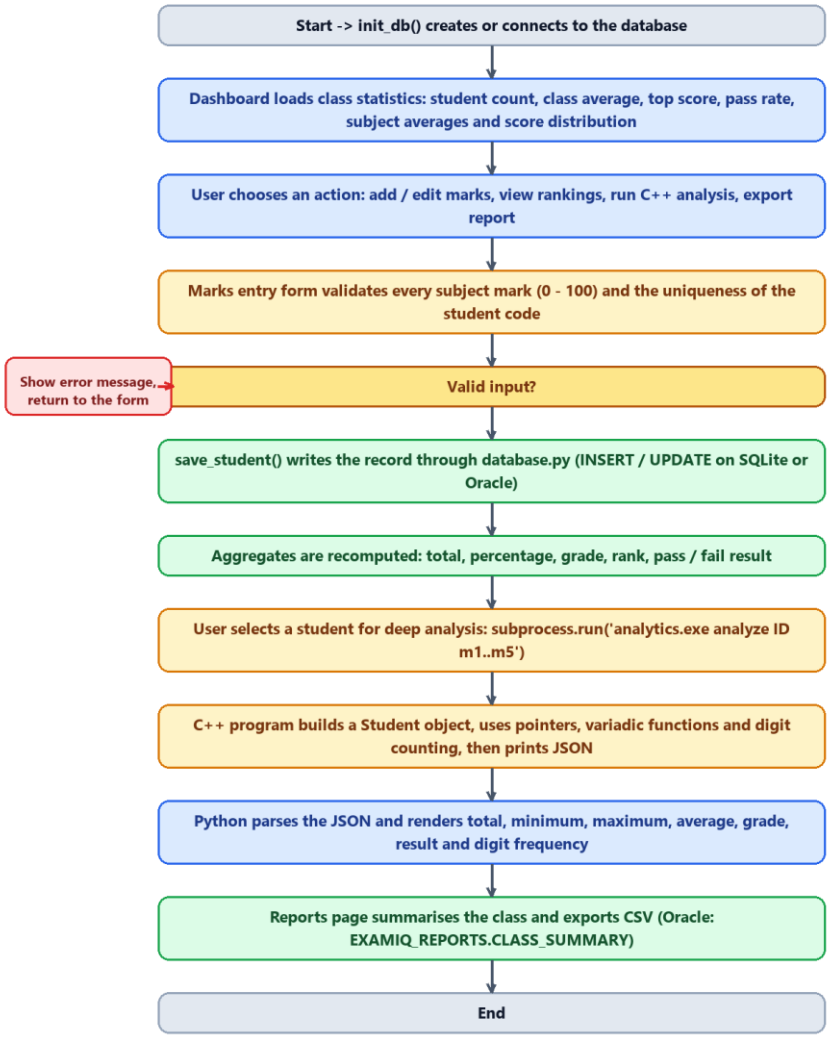


Figure 2: Process flow for storage, reporting and verification

Output & Screenshots

The deployment script was executed as the project schema user. On the three sample rows inserted at the end of sql/oracle_setup.sql, the verification block returns the following values:

Listing 7: DBMS_OUTPUT of the verification block on the three sample rows

```
Students            : 3
Average percentage  : 76.67
Top total           : 437
Pass rate           : 100%
DBMS average        : 79
```

The three sample records score 437 (87.4%), 405 (81.0%) and 308 (61.6%) respectively, which gives the average percentage of 76.67, the highest total of 437 and a pass rate of 100% because all three percentages are above 40. The DBMS average is (95 + 79 + 63) / 3 = 79.

The screens below show the same data being consumed by the application, whose adapter can address either this schema or the local SQLite file.

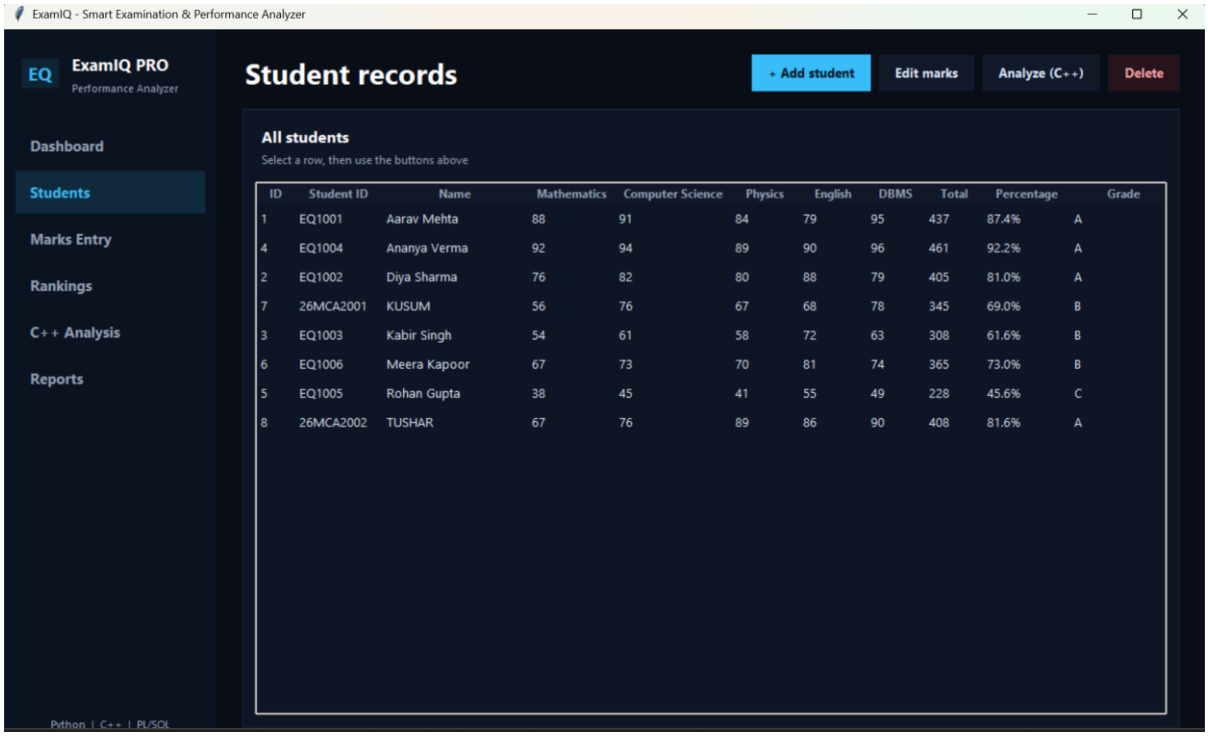


Figure 3: Records retrieved through the adapter and displayed by the desktop client

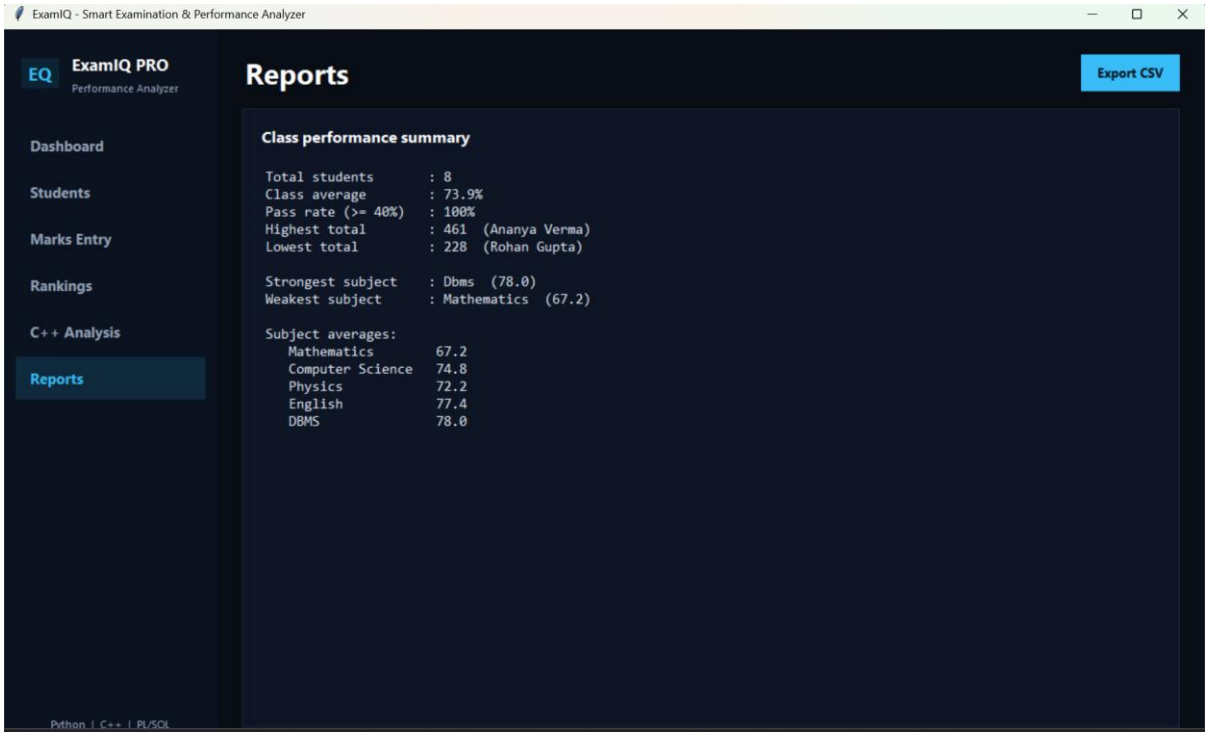


Figure 4: Reports page - the figures CLASS_SUMMARY returns, with CSV export

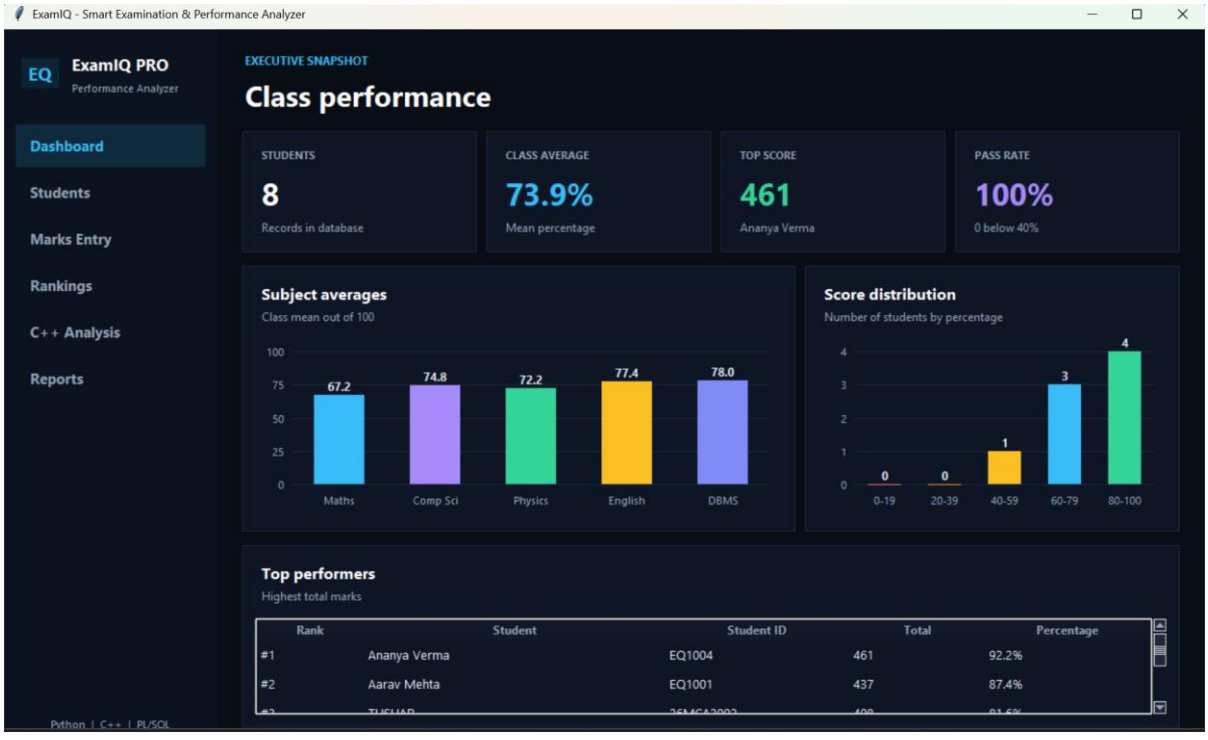


Figure 5: Dashboard aggregates computed from the stored marks

GitHub Upload

The deployment script, the adapter and the complete application are published in the following repository:

<<GITHUB REPOSITORY LINK>>

- sql/oracle_setup.sql - table, view, package, sample rows and the verification block.
- database.py - the dual-mode adapter with bind variables and explicit commits.
- tk_app.py and app.py - the clients that consume the data.
- cpp/analytics.cpp - the compiled analytics engine of the project.
- README.md - instructions for Oracle mode, including the required environment variables.

Conclusion

The PL/SQL layer gives ExamIQ a persistent, self-protecting store of examination data. The table guarantees the shape and the domain of every row, the view publishes the derived total, percentage and grade as a single definition, and the package exposes the class-level reporting as a reusable API whose aggregation runs inside the server. Because the rules live in the database, the desktop client, the web variant and any manual query all produce identical results, and because every statement from the application uses bind variables and an explicit commit, the data can be trusted as well as read.

The exercise also made the trade-off between PL/SQL and application code concrete. Set-based aggregation inside the server is clearly superior for reporting, while row-level validation and presentation logic belong in the client. Keeping the two apart - constraints and reporting in Oracle, interaction and formatting in Python - produced a system that is short, verifiable and easy to demonstrate from SQL Developer alone.

Future Advancements

- **Historical terms and audit table:** add a term column plus an audit table populated by a trigger on INSERT, UPDATE and DELETE, so that every mark change is traceable to a user and a timestamp.
- **More reporting procedures:** extend EXAMIQ_REPORTS with a per-subject pass percentage, a topper list and a weak-student report that return ref cursors, which the client can iterate directly.
- **Pipelined and analytic functions:** use RANK() and DENSE_RANK() analytic functions inside the view so that the class rank is computed by the server, and a pipelined table function for large reports.
- **Access control and least privilege:** create a reporting role with SELECT-only grants on the view and EXECUTE on the package, so that the application user never needs direct table privileges.
- **Scheduled processing:** use DBMS_SCHEDULER to compute and store a nightly class summary, turning the on-demand procedures into a small analytics warehouse.
- **Performance hardening:** add an index on student_code, review the execution plans of the reporting queries, and enable PL/SQL native compilation and result caching for the package.